# Ticket Booking System

## Testable Design, TDD, and Mocking Verification

Hashemite University
Software Testing

1. Section One: Dependency Injection Analysis

   In our project, we chose to use Constructor Dependency Injection for the BookingManager. Instead of creating the payment or notification objects inside the class, we pass them as parameters (IPaymentGateway, INotificationService, and IEventRepository). This makes the code much easier to test because we can 'inject' Mock objects instead of real services. This way, our unit tests focus only on the booking logic itself, and we don't have to worry about whether the database or the internet is working during the test.

2. Section two: Limitation of Mocking

   Using Mockito for mocking is great for isolation, but it has some risks. The main issue is what is called a 'False Sense of Security'. Since we program the mock to return exactly what we want (like a 'Success' message), the test will pass even if the real system has a bug. For example, if the actual Payment Gateway is down, the unit test will not catch that because it is only talking to a mock. That is why unit tests are not enough we still need Integration Testing to see how everything works together in the real world.

3. Section3: Team Contribution Matrix.

| Student Name & ID | Task & Responsibilities |
|---|---|
| Marwa Abu Al-Haj (2338029) | Implemented the BookingManager class using Dependency Injection, and prepared the final project report. |
| Weam Aldaja (2337945) | Developed the unit tests using JUint and Mockito, created the mock objects for dependencies, and verified the booking logic scenarios. |